

JALON 4 – Conception de l'Application & Architecture

Projet : CALENDRIA

Étape : Avril (Jalon 4)

Livrables : Conception technique globale de l'application (Dossier UML + Description Structurale).

Table des matières

- 1. [Introduction](#)
- 2. [Vue Fonctionnelle \(Use Cases\)](#)
- 3. [Vue Dynamique \(Séquences\)](#)
- 4. [Vue Statique \(Classes\)](#)
- 5. [Vue Structurale \(Architecture N-Tiers\)](#)

Introduction

Ce dossier (correspondant aux **chapitres VII et VIII** du rapport final) présente la conception technique de Calendria. Il fait directement le lien entre :

- Les **besoins fonctionnels** définis dans le CDCF (Jalon 1), traduits ici en Cas d'Utilisation UML.
- Les **parcours utilisateurs** des wireframes (Jalon 2), traduits en diagrammes de séquences.
- Le **modèle physique de données** (Jalon 3), traduit en diagramme de classes et architecture logicielle.

Note sur le design des diagrammes (.puml) :

Tous les diagrammes PlantUML respectent désormais une charte graphique avec une police d'en-tête blanche sur fond sombre (Code `#2E86C1`) afin de garantir une lisibilité optimale lors de la génération (comme vu lors de la correction du Jalon 3).

Vue Fonctionnelle (Use Cases)

Fichier : `diagramme_cas_utilisation.puml`

Ce diagramme offre une vision panoramique des interactions utilisateurs-système.

- Il identifie les **3 acteurs principaux** : le Client, le Restaurateur (administrateur du dashboard) et le Système Automatique (Intelligence Artificielle et crons/tâches asynchrones).
- Les Use Cases sont groupés logiquement par packages ("Dashboard", "IA Multi-Canal" et "Traitement Automatique") conformément de l'analyse fonctionnelle.

Vue Dynamique (Séquences)

Pour illustrer les processus métier complexes, 3 scénarios clés ont été découpés en Diagrammes de Séquences.

Fichier : `sequence_reservation_chatbot.puml`

Explicite le cœur d'innovation du projet Calendria : **la création d'une réservation via IA**.

- Montre les itérations de collecte de données entre l'utilisateur, notre contrôleur et ChatGPT.
- Démontre l'interaction entre nos différentes couches de Services (`ChatbotService` , `DisponibiliteService` , `ReservationService`).

🔗 **Fichier : `sequence_authentication.puml`**

Explicite la **sécurité Stateless du Dashboard**.

- Documente l'interaction entre le Frontend React (Axios) et le module Symfony LexikJWT pour récupérer un jeton signé.

🔗 **Fichier : `sequence_gestion_reservation.puml`**

Explicite l'**Architecture API REST**.

- Montre l'utilisation du JWT pour protéger les requêtes.
- Illustre un exemple de workflow (Marquer un Noshow) provoquant des modifications en base de données complexes et un effet de bord externe (Envoi SMS Notification).

🔗 Vue Statique (Classes)

🔗 **Fichier : `diagramme_classes.puml`**

Ce diagramme va au-delà du Modèle Logique de Données du Jalon 3. Il illustre l'intégration des concepts métiers dans le code objet PHP.

- Séparé en 4 "Packages" délimitant nos couches MVC : `Entity` (Modèles ORM Doctrine), `Repository` (Accès DQL orienté objet), `Service` (Logique métier complexe), et `Controller` (Exposition API).
- Les cardinalités du Jalon 3 (MERISE) ont été traduites en multiplicités objet UML (ex: Une entité Restaurant possède une matrice d'entités Reservation).

🔗 Vue Structurale (Architecture N-Tiers)

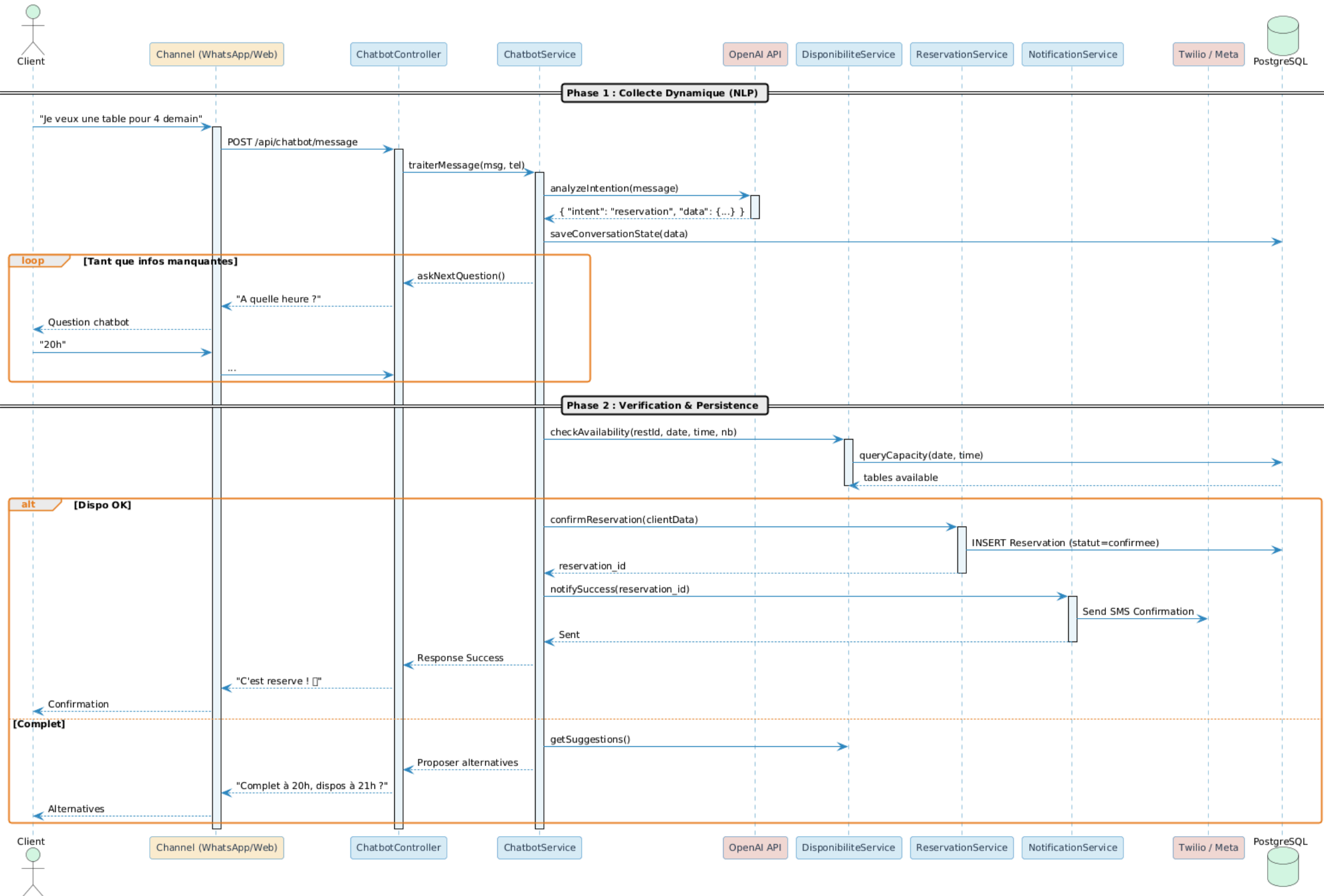
🔗 **Fichiers : `architecture_ntiers.puml` et `architecture.md`**

- Le diagramme **PUML** expose le déploiement physique : 3 Niveaux de Tiers (UI React SPA, API Symfony RESTful, BDD PostgreSQL) s'exécutant de manière isolée sur un réseau intra-Docker, tout en communiquant avec les APIs OpenAI et Twilio.
- Le document texte `architecture.md` décrit finement cette architecture. Il justifie l'utilisation des patterns de conception (MVC) et le respect des dogmes object-oriented (Principes S.O.L.I.D. comme l'injection de dépendances et la Single Responsibility) imposés par l'énoncé.

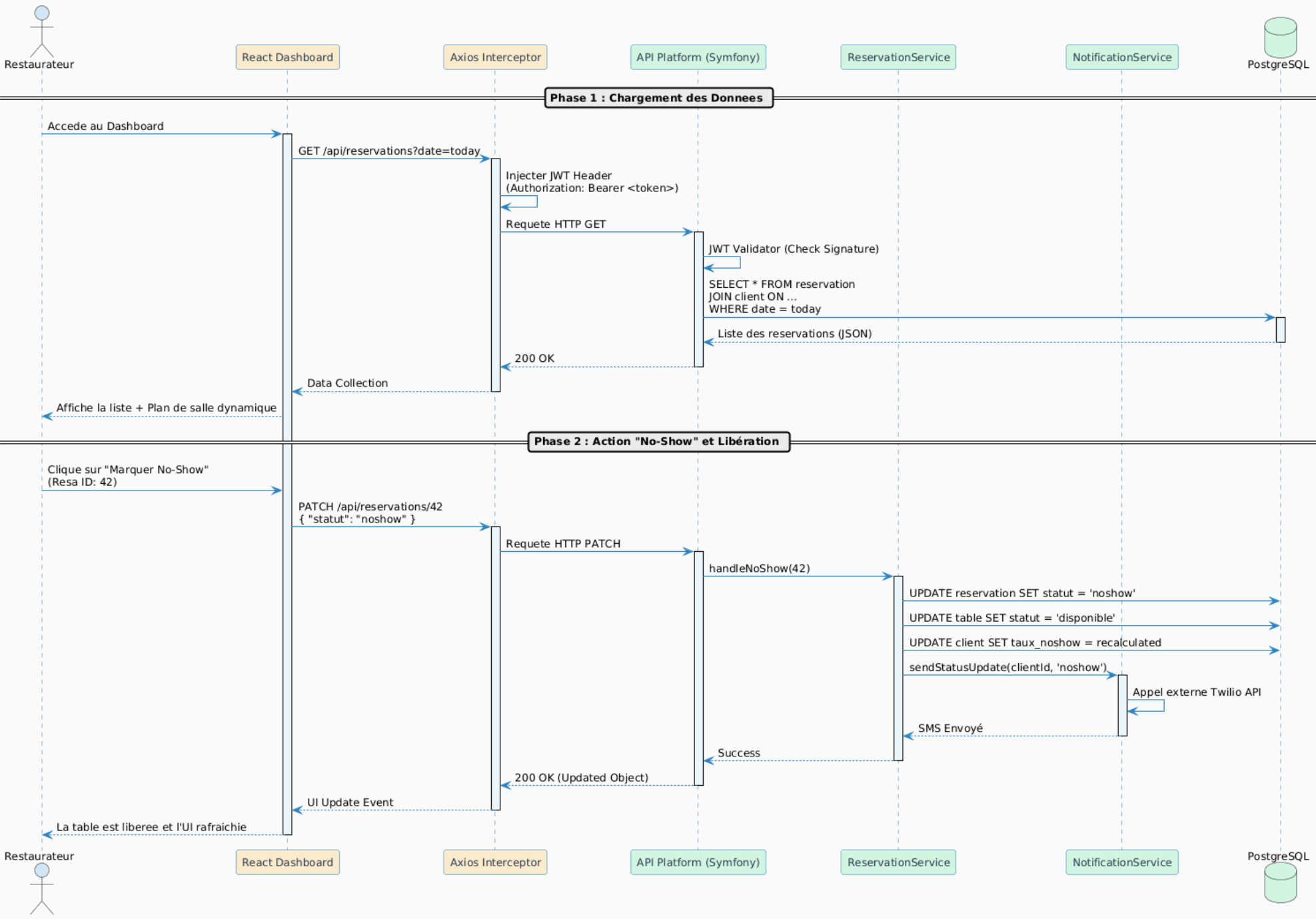
Ces documents constituent la fondation absolue pour coder l'API Backend et les composants React sereinement au **Jalon 5**.

CALENDRIA - Diagramme de Sequence

UC2 : Reservation via Chatbot IA

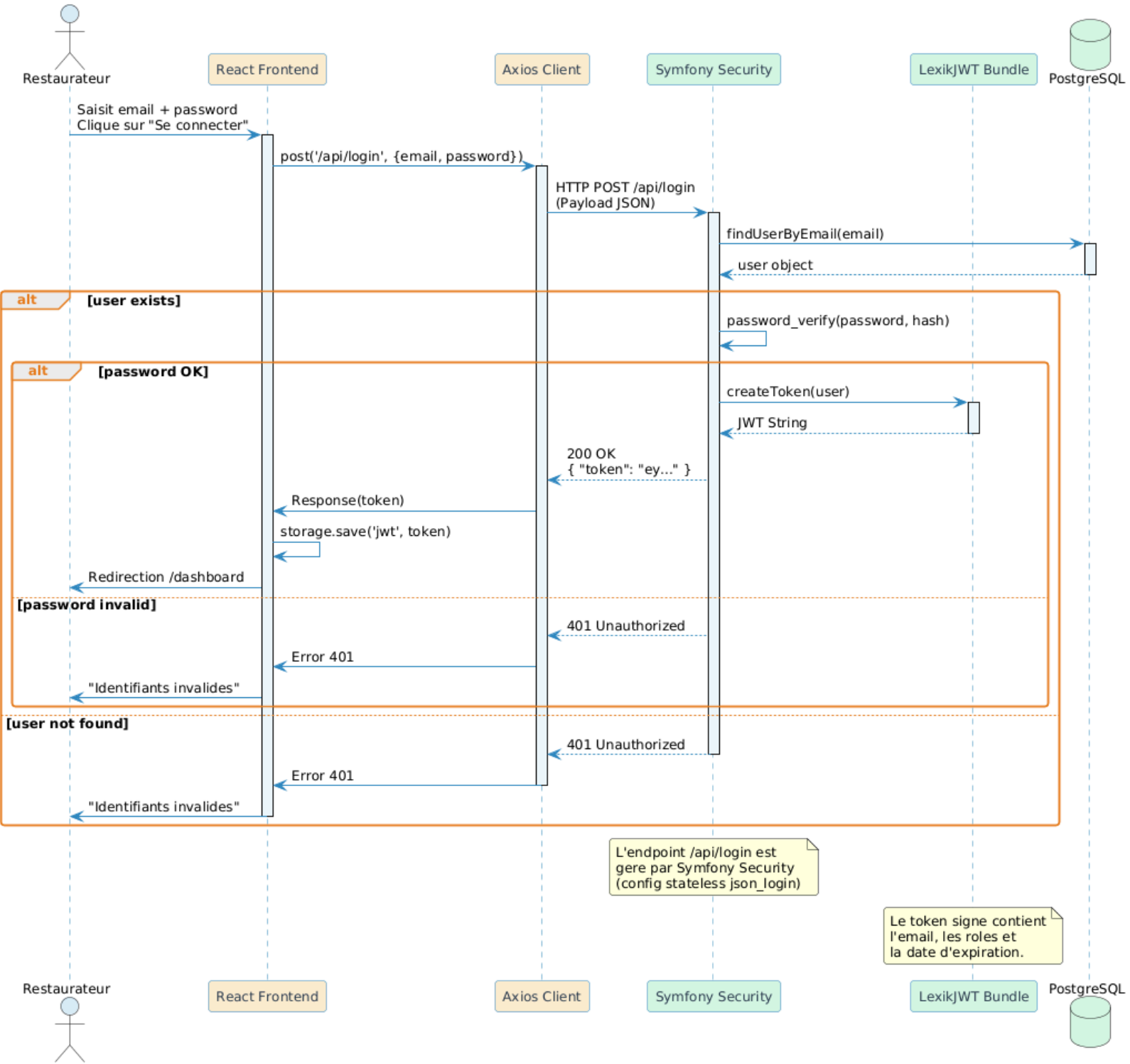


CALENDRIA - Diagramme de Sequence
UC8 & UC9 : Gestion des reservations (Dashboard)



Concepts clés :
1. Le token JWT est injecte par l'intercepteur a chaque requete.
2. L'API est protegee (Stateless) : le serveur verifie le token avant d'acceder aux DB.
3. Le changement de statut declenche des effets de bord complexes (SMS + MAJ BDD).

CALENDRIA - Diagramme de Sequence
UC6 : Authentification du Restaurateur (JWT)



CALENDRIA - Diagramme de Classes

Architecture par packages (MVC / Doctrine)

src/Controller

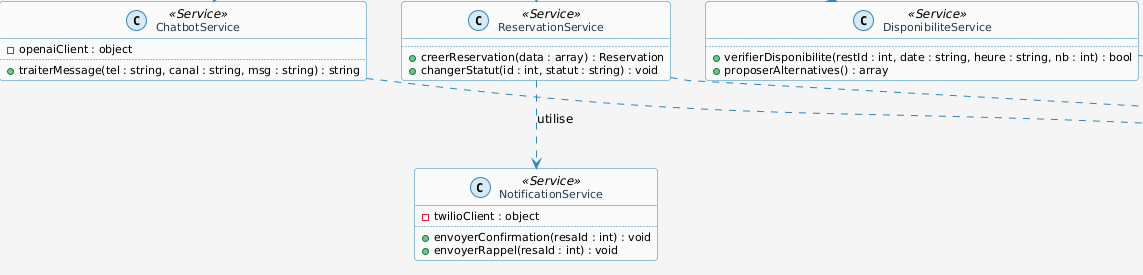


injecte

events

events

src/Service

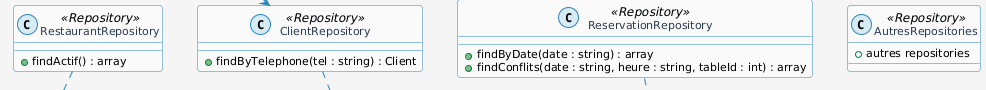


utilise

utilise

utilise

src/Repository

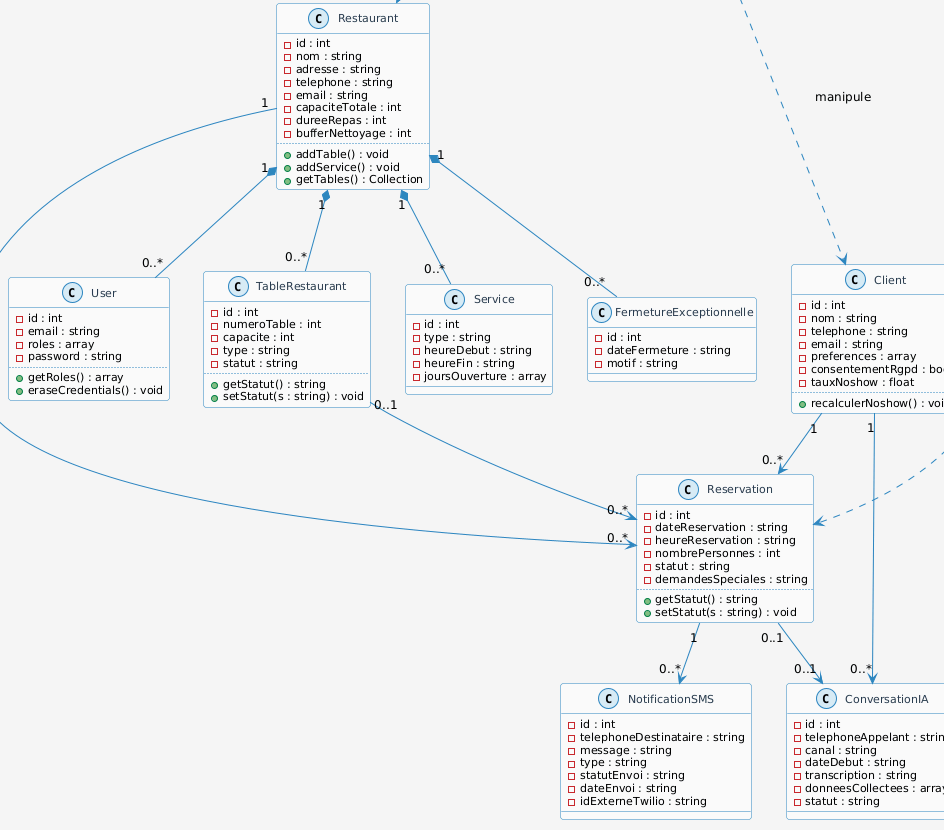


utilise

utilise

manipule

src/Entity



Architecture Multi-Couches

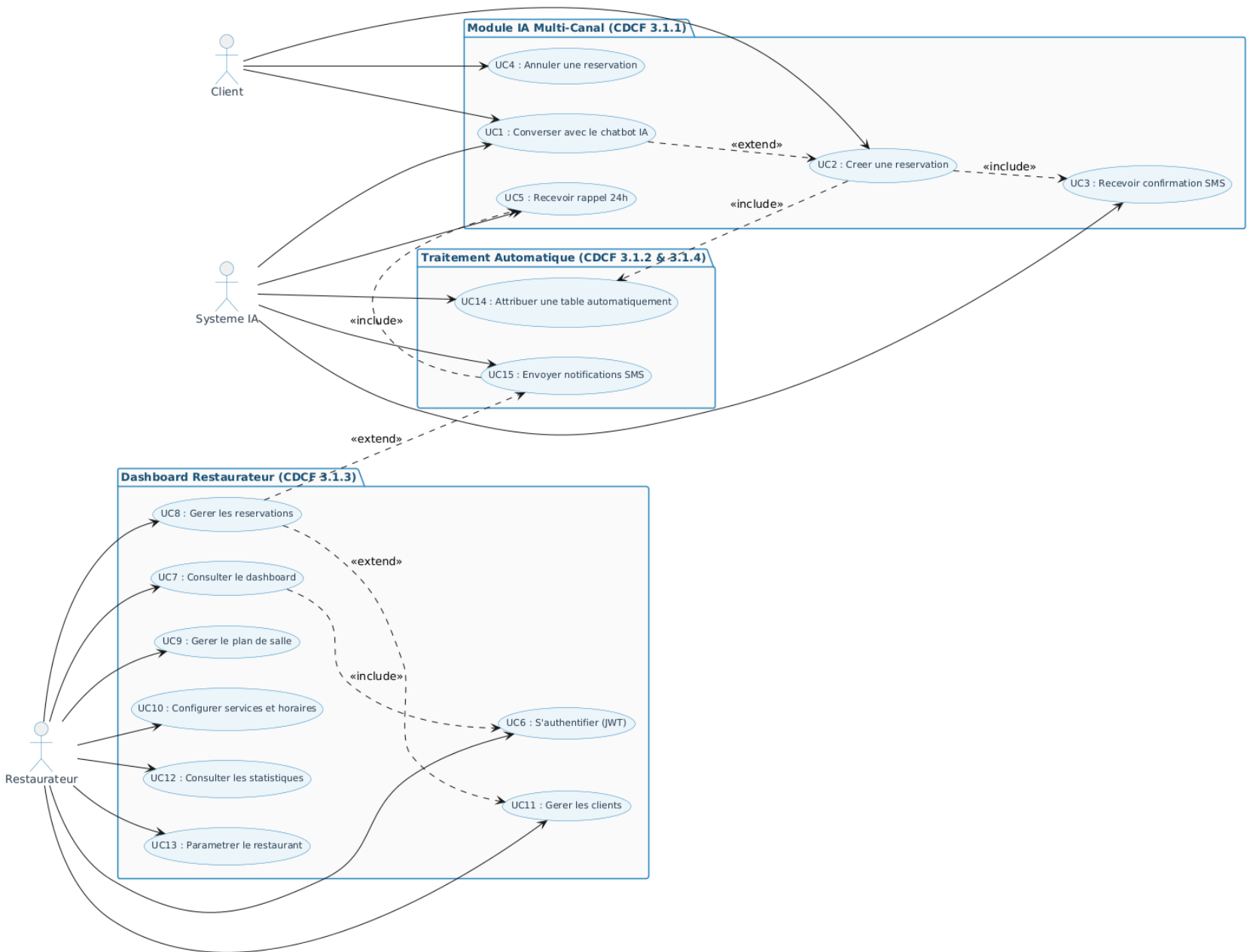
Controllers : Expose l'API REST, route les requetes

Services : Logique métier complexe (IA, Twilio, disponibilite)

Repositories : Acces exclusif BDD (requetes DQL/SQL)

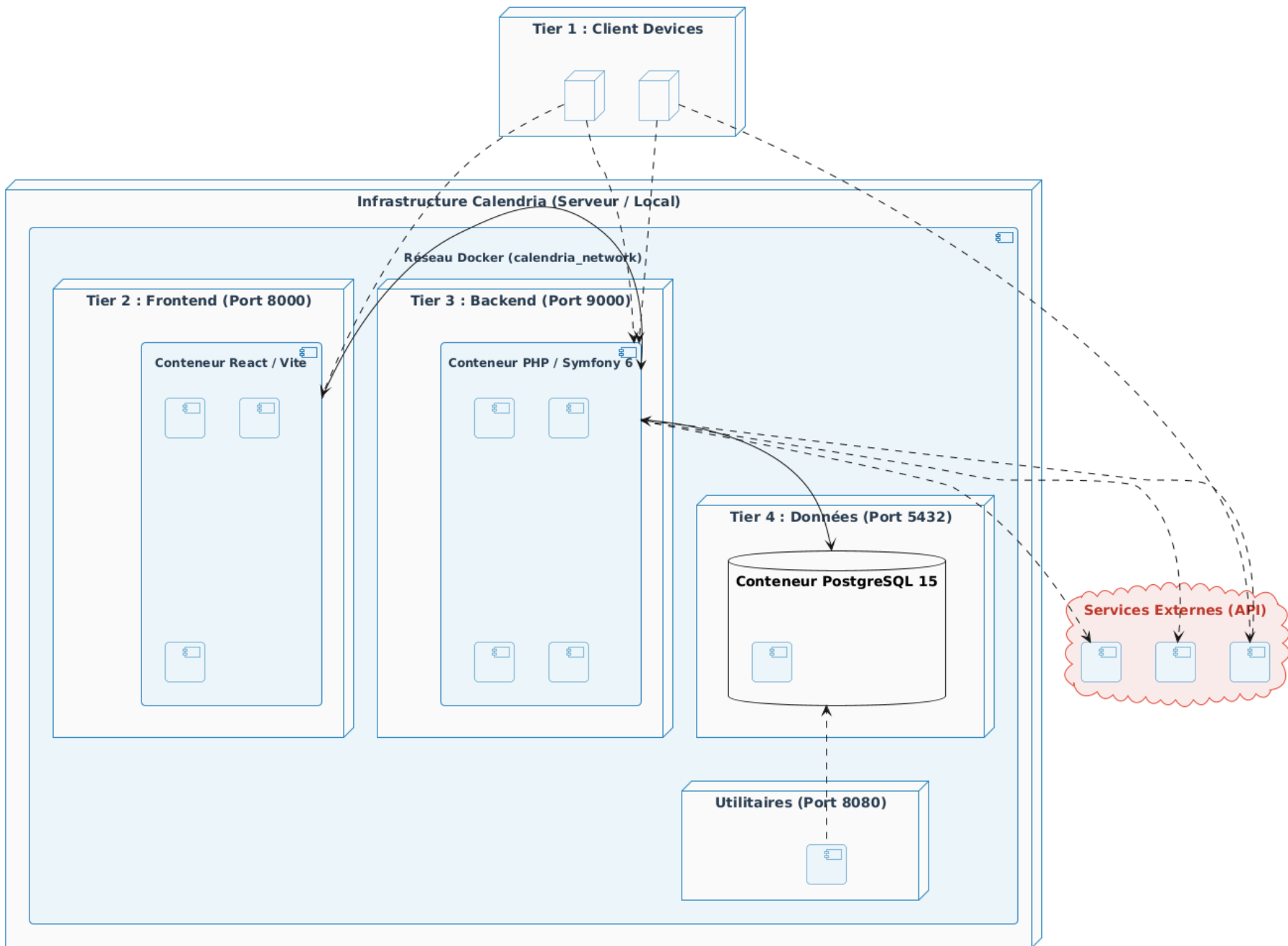
Entities : POPO mappees en base de donnees via Doctrine ORM

CALENDRIA - Diagramme de Cas d'Utilisation



Client : Reserve via chatbot (WhatsApp, Widget, SMS)
Restaurateur : Gere via le dashboard web
Systeme IA : Chatbot, tables, notifications
«include» : UC source => UC cible OBLIGATOIRE
«extend» : UC source => UC cible OPTIONNEL

CALENDRIA - Architecture n-tiers & Infrastructure Docker Déploiement Physique et Logique



Conteneurisation (Docker Compose) :

Séparation Front/Back :